# PurePy: A Functional Subset of Python

version 0.1.4

Nikolaus Huber[1], Dominic Orchard[2,1], Jacob Pake[2], Roly Perera[1,3]

[1] *Department of Computer Science and Technology, University of Cambridge, UK*
[2] *School of Computing, University of Kent, UK*
[3] *School of Computer Science, University of Bristol, UK*

PurePy is a pure functional subset of Python, providing a common functional language for use by researchers in programming languages and pedagogy, while providing a front-end familiar and accessible to the Python community. This specification defines a versioned standard with formal syntax, well-formedness rules based on a definite assignment analysis, and an operational semantics intended to preserve the meaning of well-formed PurePy programs in Python. By combining functional programming with Python-compatible surface syntax, PurePy aims to support portable implementations and lower-friction adoption of new language ideas for modelling, data processing, analysis, and visualisation.

## 1 Introduction

- Define subset of Python called PurePy, defined as a syntactic subset further restricted by a well-formedness predicate
- Define a semantics for PurePy such that the semantics of a PurePy program is identical to its semantics in Python
- Some well-formedness and evaluation rules could be given by a desugaring, but for now we avoid having two separate syntaxes

## 2 Syntax

To do:
- Reserved words
- Operators and precedence
- Expression grammar
- Recursion and mutual recursion
- Operational semantics of `if` statements
- Data classes and `match`

### 2.1 Contexts

Contexts $\Gamma, \Delta$ are finite maps from sets of variables to a *definite assignment* status chosen from the 2-point Boolean lattice $\{\top, \bot\}$, where $\top$ indicates *definitely assigned* and $\bot$ indicates *not definitely assigned*. (Or think of these as degenerate types "any" and "none".) The definite assignment terminology is borrowed from Java.

**Context operations.** Contexts admit a non-commutative monoid of sequential composition $\Gamma \cdot \Delta$ ("override"), defined as right-biased map union with $\varnothing$ as left and right unit, and an idempotent commutative semigroup of parallel composition $\oplus$ ("merge") defined as follows:

$$\Gamma \oplus \Delta := \{x : \bot \mid \forall x \in (\Gamma \setminus \Delta) \cup (\Delta \setminus \Gamma)\} \cup \{x : \Gamma(x) \wedge \Delta(x) \mid x \in \Gamma \cap \Delta\}$$

where $\mathrm{dom}(\Gamma \oplus \Delta) = \mathrm{dom}(\Gamma) \cup \mathrm{dom}(\Delta)$. The $\oplus$ operator lacks a unit because the neutral element for any $\Gamma$ has the same domain as $\Gamma$.

**Lemma 1** (Distributivity)**.**

$$(\Delta \oplus \Delta') \cdot \Gamma = (\Delta \cdot \Gamma) \oplus (\Delta' \cdot \Gamma).$$

Context operations $\cdot$ and $\oplus$ lift to a partial semiring over result types $R$, with $\mathsf{Assigns}\ \varnothing$ as left and right unit for $\cdot$ and $\mathsf{Returns}$ as unit for $\oplus$ and zero for $\cdot$. Sequential composition is defined only when the left-hand result type is not $\mathsf{Returns}$.

**Definition 1** (Partial semiring of result types)**.**

$$\begin{aligned}
\mathsf{Assigns}\ \Delta \oplus \mathsf{Assigns}\ \Delta' &= \mathsf{Assigns}\ (\Delta \oplus \Delta') \\
\mathsf{Assigns}\ \Delta \oplus \mathsf{Returns} &= \mathsf{Assigns}\ \Delta \\
\mathsf{Returns} \oplus \mathsf{Assigns}\ \Delta &= \mathsf{Assigns}\ \Delta
\end{aligned}$$

$$\begin{aligned}
\mathsf{Assigns}\ \Delta \cdot \mathsf{Assigns}\ \Delta' &= \mathsf{Assigns}\ (\Delta \cdot \Delta') \\
\mathsf{Assigns}\ \Delta \cdot \mathsf{Returns} &= \mathsf{Returns}
\end{aligned}$$

**Lemma 2** (Distributivity)**.** *When the relevant terms are defined (i.e. neither $R$ nor $R'$ is* $\mathsf{Returns}$*):*

$$(R \oplus R') \cdot S = (R \cdot S) \oplus (R' \cdot S)$$

## 2.2 Well-formedness

Every well-formed statement either *definitely returns*, or *definitely assigns* a set of variables $\Delta$, which may be empty. The base cases are that `return e` definitely returns, and `x = e` definitely assigns $\{$`x`$\}$.

## 2.3 Examples

### 2.3.1 Assignment as definition

Although PurePy is pure and all variable usage is well-scoped, local variables are introduced using an *assignment statement*, for compatibility with Python. If a variable is definitely assigned in any branch of a conditional, then for that variable to be usable after the conditional, every other branch of the conditional must either definitely return, or definitely assign that variable too. Thus the following are both well-formed:

$$
\begin{array}{rcll}
s & ::= & \texttt{pass} & \text{(Pass)} \\
& | & x = e & \text{(Assign)} \\
& | & s_1\ s_2 & \text{(Seq)} \\
& | & \texttt{return} & \text{(ReturnNone)} \\
& | & \texttt{return}\ e & \text{(Return)} \\
& | & \texttt{if}\ e\texttt{:}\ s\ \texttt{elif}\ e_1\texttt{:}\ s_1\ \cdots\ \texttt{elif}\ e_n\texttt{:}\ s_n & \text{(If)} \\
& | & \texttt{if}\ e\texttt{:}\ s\ \texttt{elif}\ e_1\texttt{:}\ s_1\ \cdots\ \texttt{elif}\ e_n\texttt{:}\ s_n\ \texttt{else}\texttt{:}\ s' & \text{(IfElse)} \\
& | & \texttt{def}\ f(\vec{x})\texttt{:}\ s & \text{(Def)} \\
\\
e & ::= & x & \text{(Var)} \\
\\
b & ::= & \top & \text{(Definitely assigned)} \\
& | & \bot & \text{(Not definitely assigned)} \\
\\
\Gamma, \Delta & ::= & \{x \vec{:} b\} & \\
\\
R, S & ::= & \mathsf{Returns} & \\
& | & \mathsf{Assigns}\ \Delta &
\end{array}
$$

Figure 1: Syntax of statements, expressions and contexts

```
if a < b:
   comp = "smaller"
else:
   comp = "bigger"
return comp
```

```
if a < b:
   return "smaller"
else:
   comp = "bigger"
return comp
```

The definite assignment set for a conditional that has at least one branch which definitely assigns is calculated by merging the definite assignment sets for all branches which definitely assign, using the $\oplus$ operator on environments.

### 2.3.2 Local assignment in conditional clauses

For flexibility, we permit *branch-local* variables. The assignment of `x` below (left) is permitted, and can be used locally in that branch of the conditional:

```
def foo():
  z = 6
  if b:
    x = 3
    y = x + 1
  else:
    y = 0
  return z
```

(a) Branch-local definition of `x`

```
def foo():
  x = 6
  if b:
    x = 3
    y = x + 1
  else:
    y = 0
  return x # error: partially defined
```

(b) Now with use of `x` after the conditional

$\Gamma \vdash s : R$

pass-assigns
$$\frac{}{\Gamma \vdash \texttt{pass} : \textsf{Assigns}\ \varnothing}$$

assign-assigns
$$\frac{\Gamma \vdash e}{\Gamma \vdash x = e : \textsf{Assigns}\ \{x\}}$$

return-none-returns
$$\frac{}{\Gamma \vdash \texttt{return} : \textsf{Returns}}$$

return-returns
$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{return}\ e : \textsf{Returns}}$$

seq
$$\frac{\Gamma \vdash s_1 : R \qquad \Gamma \cdot \Delta_1 \vdash s_2 : S}{\Gamma \vdash s_1\ s_2 : R \cdot S}$$

if
$$\frac{\Gamma \vdash \vec{e} \qquad \Gamma \vdash \vec{s} : \vec{R}}{\Gamma \vdash \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \cdots\ \texttt{elif}\ e_{n+1} :\ s_{n+1} : R_1 \oplus \ldots R_{n+1} \oplus \ldots \oplus \textsf{Assigns}\ \varnothing}$$

if-else
$$\frac{\Gamma \vdash \vec{e} \qquad \Gamma \vdash \vec{s} : \vec{R}}{\Gamma \vdash \texttt{if}\ e_1 :\ s_1\ \texttt{elif}\ e_2 :\ s_2\ \cdots\ \texttt{elif}\ e_{n+1} :\ s_{n+1}\ \texttt{else} :\ s_{n+2} : R_1 \oplus \cdots \oplus R_{n+2}}$$

def
$$\frac{\Gamma, x_1 : \top, \ldots, x_n : \top \vdash s : R}{\Gamma \vdash \texttt{def}\ f(\vec{x}) :\ s : \textsf{Assigns}\ \{f\}}$$

(a) Statement $s$ is well-formed in $\Gamma$

$\Gamma \vdash e$

$$\frac{x : \top \in \Gamma}{\Gamma \vdash x}\ \textsc{var}$$

(b) Expression $e$ is well-formed in $\Gamma$

However such variables only appear to be branch-local. The definition of $\oplus$ actually ensures that a variable assigned in any branch is considered assigned by the conditional; however unless it is assigned in *every* branch that does not definitely return, it will be bound to $\bot$ in the corresponding environment. Any attempt to refer to such a variable is illegal, as it refers to a variable which may not be assigned. (Thus the right-hand program is ill-formed, despite the enclosing `x = 6`, because in the `return x` statement, the inner assignment shadows the the outer one, and the inner assignment is only partial.) If we were to allow the program on the right, there would be no straightforward way to assign it a semantics which is both pure, and in agreement with Python, since in Python the function returns `3` in the case where `b` is `True`.

### 2.3.3 Unreachable code

A non-empty sequence of statements returns only if the last statement returns, and all preceding statements assign. Thus the following is illegal:

```
def foo(x):
  return x + 1
  return x    # error: unreachable
```

### 2.3.4 Conditionals without `else` clauses

An `if` statement without an `else` clause is equivalent to an `if` statement with an `else` clause whose body is a `pass` statement; such a statement never returns.

### 2.3.5 Implicit `None`

A `return` statement without an expression is equivalent to `return None`.

### 2.3.6 Implicit `return`

A function whose body $s$ assigns rather than returns is equivalent to a function whose body is $s$ followed by `return None`. This ensures every function returns, even if it is not explicitly specified.

## 3 Operational semantics

For a statement $s$, the result $\mathsf{assigns}\ \rho$ represents *normal completion* of $s$, with accumulated environment $\rho$; the result $\mathsf{returns}\ v$ represents *abrupt completion* of $s$ with return value $v$. Composing statements sequentially composes their results according to the following partial monoid, analogous to sequential composition of result types:

$$\mathsf{assigns}\ \rho \cdot \mathsf{assigns}\ \rho' = \mathsf{assigns}\ (\rho \cdot \rho')$$
$$\mathsf{assigns}\ \rho \cdot \mathsf{returns}\ v = \mathsf{returns}\ v$$

with $\mathsf{assigns}\ \varnothing$ as left and right unit. Function definitions delimit the scope of these effects, so as black boxes functions are always pure.

$$
\begin{array}{rcl}
r & ::= & \mathsf{assigns}\ \rho \\
 & | & \mathsf{returns}\ v
\end{array}
$$

(a) Evaluation results

$s \Rightarrow r$

eval-pass

$$\frac{}{\rho, \texttt{pass} \Rightarrow \mathsf{assigns}\ \{\varnothing\}}$$

eval-assign

$$\frac{\rho, e \Rightarrow v}{\rho, x = e \Rightarrow \mathsf{assigns}\ \{x \mapsto v\}}$$

eval-return

$$\frac{\rho, e \Rightarrow v}{\rho, \texttt{return}\ e \Rightarrow \mathsf{returns}\ v}$$

eval-seq

$$\frac{\rho, s_1 \Rightarrow r \qquad \rho \cdot \rho_1, s_2 \Rightarrow r'}{\rho, s_1\ s_2 \Rightarrow r \cdot r'}$$

eval-def

$$\frac{}{\rho, \texttt{def}\ f(x):\ \ s \Rightarrow \mathsf{assigns}\ \{f \mapsto \langle \rho, s \rangle\}}$$

Figure 5: Statement evaluation

$\rho, e \Rightarrow v$

$$\frac{}{\rho, x \Rightarrow \rho(x)}\ \textsc{var}$$

Figure 6: Expression evaluation